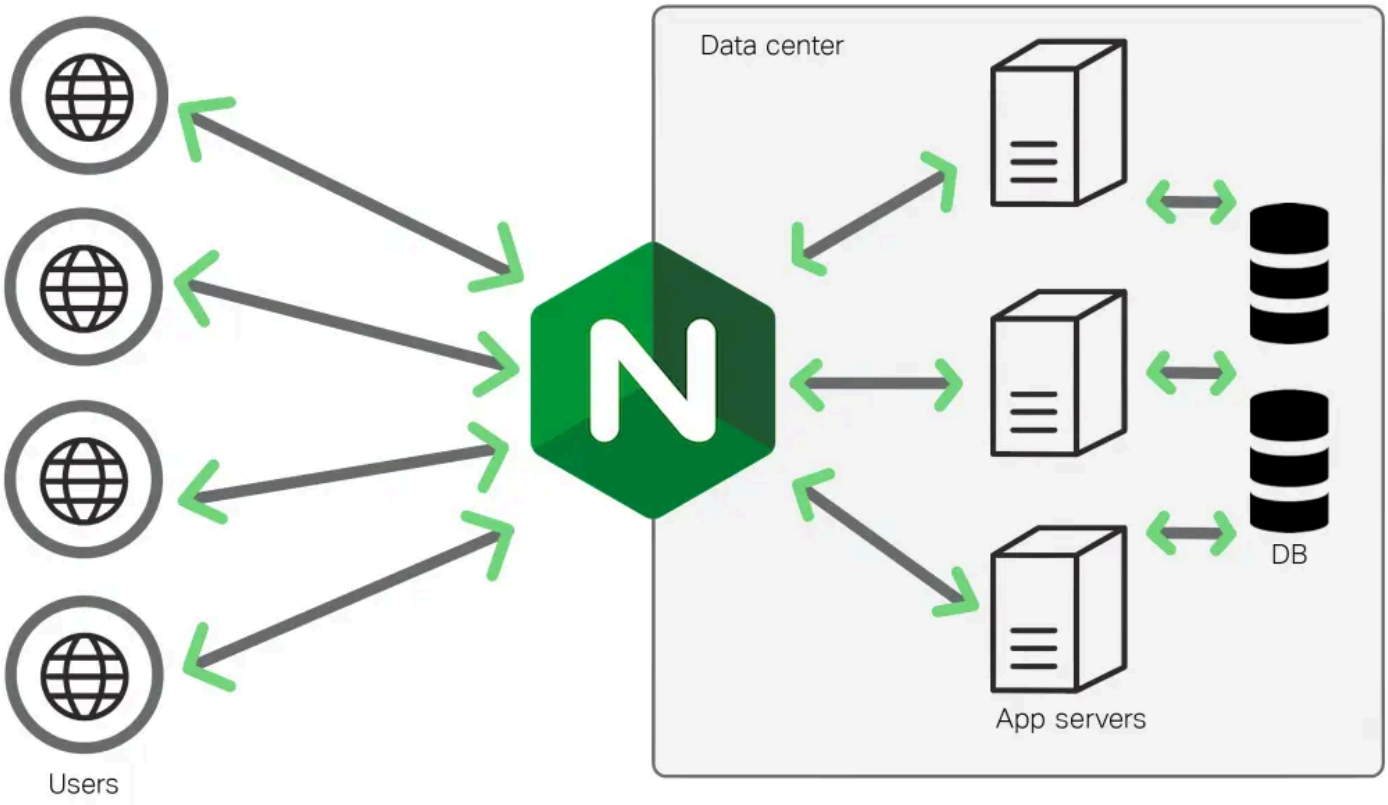
Day 3

NGINX Web Server



This training provides a foundational guide for setting up and managing NGINX — a high-performance web server, reverse proxy, and load balancer. You will install, configure, host websites, set up reverse proxying with Docker, and troubleshoot common issues.

#	Topic	Description
01	**Introduction to NGINX**	Architecture, roles: web server, reverse proxy, load balancer
02	**Installation & Commands**	APT install, systemctl management, config testing
03	**Configuration Structure**	Contexts, directives, key directories
04	**Hosting a Basic Website**	Web root, permissions, server blocks, enabling sites

05	**Nginx vs Apache**	Comparison, use cases, VirtualHost vs site blocks
06	**Reverse Proxy with Docker**	proxy_pass, Docker backend, /etc/hosts local DNS
07	**Practical: Multiple Sites**	Hosting app1.local and app2.local with site blocks
08	**Troubleshooting**	Logs, status checks, common errors

1. Introduction to NGINX

NGINX uses an asynchronous, event-driven architecture, allowing it to handle thousands of concurrent connections with minimal resource usage. Unlike traditional thread-per-connection servers, NGINX uses a small, fixed number of worker processes that handle all requests via non-blocking I/O.

Role	Description	Example Use Case
Web Server	Serves static content (HTML, CSS, images) directly from disk with high efficiency.	Hosting a company website, documentation site, or landing page.
Reverse Proxy	Acts as an intermediary: receives client requests and forwards them to backend apps.	Fronting a Node.js, Python (Flask/Django), or PHP application running in Docker.
Load Balancer	Distributes incoming traffic across multiple backend servers to prevent overload.	Spreading API requests across 3 backend containers to ensure high availability.

2. Installation and Basic Commands

On Ubuntu/Debian systems, install NGINX using the APT package manager:

```
sudo apt update
sudo apt install nginx -y

# Verify the installation
nginx -v
sudo systemctl status nginx
```

Essential Management Commands

Command	What It Does	When to Use

sudo systemctl start nginx	Starts the NGINX service.	After install or if the service is stopped.
sudo systemctl stop nginx	Stops the NGINX service completely.	For maintenance or when switching configs.
sudo systemctl restart nginx	Stops then starts NGINX (brief downtime).	After major config changes that require a full reset.
sudo systemctl reload nginx	Reloads config without dropping live connections.	For day-to-day config changes — zero downtime.
sudo nginx -t	Tests nginx.conf for syntax errors without applying changes.	Always run before reload/restart to catch mistakes.
sudo systemctl enable nginx	Enables NGINX to start automatically at boot.	After first install.

3. Configuration Structure

NGINX configuration is hierarchical, built from directives (key-value instructions) and contexts (blocks that group related directives). Understanding this hierarchy is essential before editing any config file.

Context	Scope	Example Directives
main	Global settings — applies to the entire NGINX process.	worker_processes, error_log, pid
events	Controls connection handling behaviour.	worker_connections, use epoll
http	All HTTP/HTTPS traffic settings. Parent of server blocks.	access_log, gzip, keepalive_timeout
server	Settings for one virtual host / domain.	listen, server_name, root, index
location	URL path-specific rules within a server block.	try_files, proxy_pass, return

Key Directories

Path	Purpose
/etc/nginx/nginx.conf	The primary configuration file. Contains the main, events, and http contexts.
/etc/nginx/sites-available/	Where all per-site configuration files are stored (enabled or not).

/etc/nginx/sites-enabled/	Contains symlinks to active configs in sites-available. NGINX reads from here.
/var/www/html/	Default web root. Replace or add new directories for each hosted site.
/var/log/nginx/access.log	Logs every incoming request: IP, status code, URL, response size.
/var/log/nginx/error.log	Logs errors: config problems, permission issues, upstream failures.

4. Hosting a Basic Website

Follow these steps to host a custom domain (e.g., example.com) on NGINX:

Step 1 Create the Web Root Directory

```
sudo mkdir -p /var/www/example.com/html
sudo chown -R $USER:$USER /var/www/example.com/html
sudo chmod -R 755 /var/www/example.com

# Create a basic index page
echo '<h1>Welcome to example.com</h1>' > /var/www/example.com/html/index.html
```

Step 2 Create the Server Block Config

Create the file /etc/nginx/sites-available/example.com:

```
server {
    listen 80;
    server_name example.com www.example.com;

    root  /var/www/example.com/html;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Step 3	Enable the Site and Reload NGINX

```
# Create symlink to sites-enabled
sudo ln -s /etc/nginx/sites-available/example.com \
           /etc/nginx/sites-enabled/
```

```
# Always test config before reloading
sudo nginx -t
sudo systemctl reload nginx
```

> try_files $uri $uri/ =404 tells NGINX to first look for an exact file match, then a directory, and return a 404 if neither is found. This is the standard pattern for static sites.

5. Nginx vs Apache

Nginx and Apache are the two most widely deployed web servers. They differ significantly in architecture and are suited to different workloads. Understanding their differences helps you choose the right tool for each job.

Architecture	Event-driven, async — one worker handles thousands of connections.	Process/thread per connection (prefork or worker MPM).
Static Files	Extremely fast; serves files directly from disk with minimal overhead.	Good, but slower than Nginx for high concurrency.
Dynamic Content	Delegates to backend (PHP-FPM, Node.js, Python) via proxy_pass.	Handles PHP natively via mod_php inside the process.
Config Style	Server blocks in sites-available; symlinked to sites-enabled.	VirtualHost blocks; enabled with a2ensite / a2dissite.
Use Case	High-traffic static sites, reverse proxy, API gateway, Docker frontends.	WordPress, Drupal, PHP apps needing .htaccess overrides.
.htaccess	Not supported — all config must be in server/location blocks.	Fully supported; allows per-directory config overrides.
Modules	Compiled in at build time or loaded as dynamic modules.	Dynamically loadable .so modules; extensive module library.

Config Examples Side by Side

Nginx — Server Block	Apache — VirtualHost
server { listen 80; server_name myapp.local;	<VirtualHost *:80> ServerName myapp.local DocumentRoot /var/www/myapp

```
    root /var/www/myapp;
    index index.html;
    location / {
        try_files $uri =404;
    }
}
```

```
    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/\
        access.log combined
</VirtualHost>
```

6. Reverse Proxy with Docker

A reverse proxy is a server that receives client requests and forwards them to a backend service. NGINX is commonly used to front applications running inside Docker containers, keeping them off public ports while managing routing, headers, and logging centrally.

Client Browser	→	**Nginx :80(Reverse Proxy)**	→	**Docker Containernginx:alpine on :8080**

Step 1	**Deploy the Application in Docker**

Start a simple nginx container (acts as our sample backend app) on port 8080:

```
# Start a basic nginx backend container on port 8080
docker run -d -p 8080:80 --name nginx-container nginx:alpine

# Confirm it's running
docker ps
curl http://localhost:8080
```

Step 2	**Configure Nginx as a Reverse Proxy**

Create /etc/nginx/sites-available/myapp.local to proxy requests to the Docker container:

```
server {
    listen 80;
    server_name myapp.local;

    location / {
        proxy_pass         http://127.0.0.1:8080;  # Forward to Docker container
        proxy_set_header   Host              $host;
        proxy_set_header   X-Real-IP         $remote_addr;
        proxy_set_header   X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header   X-Forwarded-Proto $scheme;
    }
}


sudo ln -s /etc/nginx/sites-available/myapp.local /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

Step 3	Add Local DNS via /etc/hosts

The /etc/hosts file maps hostnames to IPs locally before any DNS query. Add your local domain here for development and testing:

```
# Edit /etc/hosts
sudo nano /etc/hosts

# Add this line:
127.0.0.1    myapp.local

# Test it
curl http://myapp.local
# Expected: default nginx page served from the Docker container
```

> /etc/hosts entries are local to your machine only. For team environments, consider using dnsmasq or an internal DNS server so all team members can resolve the same domain names.

7. Practical Session: Host Multiple Websites

In this practical session you will host two separate websites (app1.local and app2.local) on a single NGINX server using site blocks, each with its own web root and configuration.

Step 1	Install and Validate NGINX

```
sudo apt update && sudo apt install nginx -y
sudo systemctl start nginx
sudo systemctl enable nginx
sudo nginx -t
# Expected: syntax is ok / test is successful
```

Step 2	Create Web Root Directories

```
# Create directories for both sites
sudo mkdir -p /var/www/app1.local/html
sudo mkdir -p /var/www/app2.local/html

# Set ownership
sudo chown -R $USER:www-data /var/www/app1.local
sudo chown -R $USER:www-data /var/www/app2.local
sudo chmod -R 755 /var/www/app1.local
sudo chmod -R 755 /var/www/app2.local

# Create index pages for each site
echo '<h1>Welcome to App 1</h1>' > /var/www/app1.local/html/index.html
echo '<h1>Welcome to App 2</h1>' > /var/www/app2.local/html/index.html
```

Step 3	Create Site Block Configs

Create /etc/nginx/sites-available/app1.local:

```
server {
    listen 80;
    server_name app1.local;
```

```
    root  /var/www/app1.local/html;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Create /etc/nginx/sites-available/app2.local:

```
server {
    listen 80;
    server_name app2.local;

    root  /var/www/app2.local/html;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Step 4	Enable Both Sites

```
sudo ln -s /etc/nginx/sites-available/app1.local /etc/nginx/sites-enabled/
sudo ln -s /etc/nginx/sites-available/app2.local /etc/nginx/sites-enabled/

sudo nginx -t
sudo systemctl reload nginx
```

Step 5	Configure /etc/hosts and Test

```
# Add both domains to /etc/hosts
sudo sh -c 'echo "127.0.0.1 app1.local" >> /etc/hosts'
sudo sh -c 'echo "127.0.0.1 app2.local" >> /etc/hosts'

# Test both sites
curl http://app1.local
# Expected: <h1>Welcome to App 1</h1>

curl http://app2.local
```

```
# Expected: <h1>Welcome to App 2</h1>
```

> Public IP: If your server has a public IP, point your domain's DNS A record to that IP (replacing 127.0.0.1 in /etc/hosts). Remove the /etc/hosts entry and test from an external browser — NGINX will serve the correct site based on the server_name header.

8. Troubleshooting

Symptom	Likely Cause	Fix
502 Bad Gateway	Upstream (Docker/app) is down or wrong port in proxy_pass.	Check docker ps; verify the port in proxy_pass matches.
403 Forbidden	File/directory permissions too restrictive for www-data.	chmod 755 dirs, 644 files; chown www-data.
404 Not Found	root path is wrong or index file missing.	Double-check root directive and file exists at that path.
NGINX won't start	Config syntax error or port 80 already in use.	Run sudo nginx -t; check sudo ss -tlnp | grep :80.
Site not resolving	/etc/hosts entry missing or wrong IP.	Verify the entry: cat /etc/hosts | grep myapp.local.
Changes not applied	Forgot to reload NGINX after editing config.	Always run sudo nginx -t && sudo systemctl reload nginx.

Useful Log & Debug Commands

```
# Watch error log in real time
sudo tail -f /var/log/nginx/error.log

# Watch access log in real time
sudo tail -f /var/log/nginx/access.log

# Check NGINX service status and recent log lines
sudo systemctl status nginx

# Confirm NGINX is listening on port 80
```

```
sudo ss -tlnp | grep nginx

# Test config syntax
sudo nginx -t
```

Nginx Web Server Tasks

Complete all tasks before the next session. Each task maps to a topic covered in this guide.

T1	Test all NGINX management commands	Installation & Commands

- Install NGINX and confirm the welcome page loads at http://localhost.
- Run each management command (start, stop, restart, reload, enable) and note what happens to the service each time.
- Intentionally add a syntax error to nginx.conf (e.g., missing semicolon) and confirm nginx -t catches it before any reload.

```
# Introduce a syntax error
sudo nano /etc/nginx/nginx.conf  # remove a semicolon
sudo nginx -t  # should report: [emerg] ...

# Fix and retest
sudo nginx -t  # should report: syntax is ok
```

T2	Explore the NGINX config hierarchy	Configuration Structure

- Open /etc/nginx/nginx.conf and identify the main, events, and http contexts.
- Note the worker_processes and worker_connections values and research what changing them would do.
- List all files currently in sites-available and sites-enabled and verify which symlinks exist.

```
# View the main config
cat /etc/nginx/nginx.conf

# List enabled sites and verify symlinks
ls -la /etc/nginx/sites-enabled/

# Check current worker settings
grep -E 'worker_processes|worker_connections' /etc/nginx/nginx.conf
```

T3	Host a multi-page static site	Hosting a Website

- Create a site under /var/www/mysite.local with at least 3 pages: index.html, about.html, contact.html.
- Write a proper server block config and enable it.
- Add mysite.local to /etc/hosts and verify all 3 pages load via http://mysite.local/about.html etc.
- Test what happens when you visit a non-existent page — confirm NGINX returns a 404.

```
sudo mkdir -p /var/www/mysite.local/html
echo '<h1>Home</h1>'    > /var/www/mysite.local/html/index.html
echo '<h1>About</h1>'   > /var/www/mysite.local/html/about.html
echo '<h1>Contact</h1>' > /var/www/mysite.local/html/contact.html

# Test all pages
curl http://mysite.local
curl http://mysite.local/about.html
curl -o /dev/null -s -w '%{http_code}' http://mysite.local/missing
# Expected: 404
```

T4	Run two Docker containers and proxy each on a different path	Reverse Proxy & Docker

- Run two Docker containers: nginx:alpine on port 8081 and traefik/whoami on port 8082.
- Configure Nginx with two location blocks: /app1 proxying to :8081 and /app2 proxying to :8082.
- Verify both paths return distinct responses at http://myapp.local/app1 and http://myapp.local/app2.
- Confirm the X-Real-IP header is set correctly using the whoami endpoint.

```
docker run -d --name app1 -p 8081:80 nginx:alpine
docker run -d --name app2 -p 8082:80 traefik/whoami

# Nginx location blocks to add:
# location /app1/ { proxy_pass http://127.0.0.1:8081/; }
# location /app2/ { proxy_pass http://127.0.0.1:8082/; }

curl http://myapp.local/app1/
curl http://myapp.local/app2/
```

T5	Host app1.local and app2.local with unique content and verify isolation	Multiple Sites

- Complete the full practical session: create directories, configs, and /etc/hosts entries for app1.local and app2.local.
- Add a unique HTML page to each site so you can visually confirm you are hitting the right backend.
- Disable app2.local (remove its symlink) and confirm http://app2.local returns a 404 or connection error while app1.local still works.
- Re-enable app2.local and reload NGINX.

```
# Disable app2.local
sudo rm /etc/nginx/sites-enabled/app2.local
sudo nginx -t && sudo systemctl reload nginx

# Confirm app1 still works, app2 does not
curl http://app1.local   # Expected: 200
curl http://app2.local   # Expected: connection refused or 404

# Re-enable
sudo ln -s /etc/nginx/sites-available/app2.local /etc/nginx/sites-enabled/
sudo systemctl reload nginx
```

T6	Diagnose and fix three common NGINX errors	Troubleshooting

- Cause a 403 Forbidden: set a file to chmod 600 and try to access it — then fix it.
- Cause a 502 Bad Gateway: set proxy_pass to a port with no container running — observe and fix it.
- Cause a config error: remove a closing brace from a server block — run nginx -t to catch it, then fix it.
- For each scenario, write down which log file contained the error and what the exact error message was.

```
# Test 403
sudo chmod 600 /var/www/mysite.local/html/index.html
curl -o /dev/null -s -w '%{http_code}' http://mysite.local
sudo tail -5 /var/log/nginx/error.log

# Test 502
# Change proxy_pass to http://127.0.0.1:9999 (nothing running)
sudo systemctl reload nginx
curl -o /dev/null -s -w '%{http_code}' http://myapp.local
```